



PlaySBC

Product and Administration Guide

Features, architecture, deployment, operations, regression, and evidence

Version 2.6.0

Final public MIT engineering baseline

Contributor

Sudheer Kumar Vatrappu

Contents

Contents	2
Document Control	4
Important Product Boundary	4
Contents	4
Product Overview	5
Core Capabilities	5
Validated Endpoint Baseline	5
Public Release Gate	5
System Architecture	6
Component Responsibilities	6
Call Model	6
Protocol and Media Behavior	7
Signalling Baseline	7
Media Baseline	7
Evidence Versus Certification	7
Deployment Model Selection	8
Common Isolation And Safety Rules	8
Docker Deployment Model	9
Purpose And Prerequisites	9
Build Or Refresh Images	9
Run Fast Application Regression	9
Run The Real-Address Docker Topology	9
Verify, Inspect, And Clean Up	10
kind Canonical Regression Model	11
Purpose And Prerequisites	11
Create Or Recover The Cluster	11
Install Or Upgrade v2.6.0	11
Verify The Upgrade	12
Run All 70 Profiles	12
Run Selected Profiles	12
Roll Back And Clean Up	12
Minikube Compatibility Model	14
Purpose And Prerequisites	14
Make Images Available	14
Install Or Upgrade	14
Verify, Test, Roll Back, And Clean Up	14
Generic Helm Kubernetes Model	16
Operator Responsibilities	16

Prepare And Render Before Applying	16
Install Or Upgrade	16
Verify, Test, Roll Back, And Clean Up	16
Azure AKS Administration and Deployment Model	18
Purpose And Required Resources	18
Set Variables And Register Providers	18
Create ACR And AKS	18
Create Static Public IPs And Grant Identity Access	19
Import And Verify All Four Images	19
Install Or Upgrade AKS	19
Verify Azure Services And Runtime	20
Run AKS Regression: 12-Profile Readiness Suite	20
Credential Recovery	20
Roll Back And Clean Up	20
Local kind Real-Device Model	22
Validated Baseline	22
Determine The LAN Address And Create The Cluster	22
Install Or Upgrade	22
Verify And Configure Endpoints	23
Call And Capture Acceptance	23
Monitor, Roll Back, And Clean Up	23
Helm Configuration and Lifecycle	24
Operations and Observability	25
Image and Version Verification	25
Health and Metrics	25
Regression and Evidence Administration	26
Evidence Contract	26
Open Clickable Evidence	26
Security, HA, and Recovery	27
Security Baseline	27
HA Baseline	27
Recovery Sequence	27
Troubleshooting Guide	28
Capacity and Product Boundaries	29
What v2.6.0 Can Claim	29
What It Does Not Claim	29
Command Reference	30
Inspect Workloads	30
Restart Observability	30
Monitor Calls and Media	30

Release Artifact Verification	30
Canonical References	30

Document Control

Field	Value
Product	PlaySBC
Release	v2.6.0
Status	Final public MIT engineering baseline
Contributor	Sudheer Kumar Vatrapi
Audience	SIP engineers, platform engineers, lab administrators, and evaluators
Scope	Features, architecture, deployment, administration, operations, evidence, and limitations

Important Product Boundary

PlaySBC v2.6.0 is an engineering and interoperability lab. It is not a production-certified carrier SBC, and this guide does not make unmeasured capacity, compliance, or availability claims.

Contents

- Product overview and feature baseline
- System architecture and protocol behavior
- Docker, kind, minikube, Azure AKS, and real-device deployment models
- Helm lifecycle, operations, observability, and evidence
- Security, HA, recovery, troubleshooting, and honest capacity limits

Product Overview

PlaySBC combines SIP signalling, B2BUA routing, RTPengine media control, AI voice experiments, observability, and evidence-driven regression in one lab platform.

Core Capabilities

- SIP REGISTER and B2BUA calls over UDP, TCP, and TLS.
- Digest registration for synthetic endpoints and real devices.
- RTPengine anchoring with RTP/RTCP, G.711 transcoding, SRTP interworking, and NAT learning.
- Active-active and failure-injection lab foundations with Helm-managed Kubernetes topology.
- Rasa, STT, TTS, DTMF, and scripted AI Voice Gateway regression foundations.
- RFC 5359 call hold/resume signalling profiles.
- Prometheus metrics, Grafana dashboards, canonical ladders, logs, and combined PCAP evidence.

Validated Endpoint Baseline

OBi1022 user 1001 and Zoiper user 1002 have been registered through Azure AKS and used for two-way RTPengine-anchored audio. The local real-device lane advertises the Mac LAN address and remains isolated from the full regression cluster.

Public Release Gate

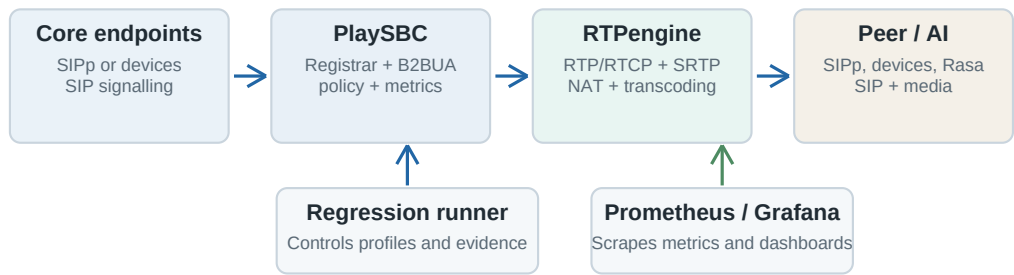
v2.6.0 contains 70 selectable Kubernetes regression profiles, live X/70 launcher progress, macOS host-sleep inhibition, and role-aware merged-PCAP certification.

System Architecture

PlaySBC separates signalling, media anchoring, test orchestration, and observability so each layer can be inspected independently.

PlaySBC component architecture

Signalling, media, orchestration, and telemetry remain independently observable



Component Responsibilities

Component	Responsibility
PlaySBC	SIP listener, registrar, B2BUA routing, dialog handling, policy, health, and metrics.
RTPengine	Media anchoring, address/port rewriting, G.711 handling, RTP/RTCP, SRTP, and NAT learning.
SIPp agents	Core and peer endpoint roles used by deterministic regression profiles.
Regression runner	Helm profile application, orchestration, verdicts, collection, and HTML/archive output.
Prometheus/Grafana	Metrics retention, dashboards, traffic, media, AI, and HA visibility.

Call Model

The inbound SIP dialog and outbound SIP dialog are separate B2BUA legs with distinct Call-IDs. v2.6.0 packet evidence checks for both roles and both INVITE legs in plain-SIP bridged profiles.

Protocol and Media Behavior

Signalling Baseline

- SIP UDP 5062, SIP TCP 5062, and SIP TLS 5061.
- REGISTER challenge and authenticated registration for configured users.
- INVITE routing to registered contacts, provisional/final responses, ACK, BYE, and CANCEL handling.
- OPTIONS health checks and transport-policy profiles.
- In-dialog hold/resume re-INVITE signalling in the public lab baseline.

Media Baseline

- RTPengine NG control on UDP 2223.
- Local and AKS real-device RTP range 30000-30049/UDP.
- Load-only regression profiles may use a larger profile-scoped RTP range.
- PCMU/PCMA G.711 operation, transcoding evidence, RTP/RTCP direction verdicts, and SRTP interworking profiles.

Evidence Versus Certification

A passing profile proves the behavior and evidence asserted by that profile. It does not automatically certify every RFC clause, endpoint, network, scale, or failure mode.

Deployment Model Selection

Choose one lane for one purpose. Do not reuse cloud values in local clusters or local real-device values in AKS.

Model	Topology	Primary use	Exposure
Docker	PlaySBC, RTPengine, SIPp containers	Fast development and dual-realm checks	Local Docker networks
kind regression	2 PlaySBC + 2 RTPengine	Canonical 70-profile regression and HA lab	Docker Desktop
Minikube	Chart compatibility topology	Kubernetes portability checks	Local NodePort/tunnel
Generic Kubernetes	Deployment or StatefulSet	Platform integration with operator-owned networking	Platform-defined
kind real device	1 PlaySBC + 1 RTPengine	LAN OBi/Zoiper calls and packet capture	Mac LAN ports
Azure AKS	1+1 readiness or active-active	Azure ACR, identity, static SIP/RTP LBs	Public/private Azure networking

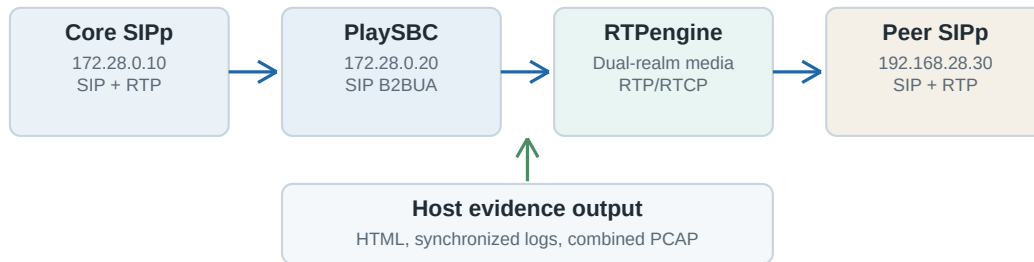
Common Isolation And Safety Rules

- Set `PLAYSBC_VERSION=2.6.0` before cloning, importing images, upgrading, or running regression.
- Use context `kind-playsbc` only for canonical local regression.
- Use context `kind-playsbc-real-device` only for LAN phone testing.
- Use the AKS context only with Azure values, ACR images, and Azure LoadBalancer resources.
- Always set both image repositories and both image tags during an upgrade. Setting only a tag can produce nonexistent defaults such as `playsbc:2.6.0` or `drachtio/rtpengine:2.6.0`.
- Verify context, Helm status, workload images, advertised addresses, and RTP range before a call or regression run.
- Use `--atomic --wait` for normal upgrades. If an interrupted operation leaves Helm pending, inspect history and roll back before starting another upgrade.

Docker Deployment Model

Docker process and topology lane

Fast development checks with isolated core and peer container networks



Purpose And Prerequisites

Docker is the fastest lane for application, signalling, media, and dual-realm topology checks. On macOS, start Docker Desktop and keep the Mac awake for load or soak work.

```

cd /Users/sudheerkumar/Documents/Codex/2026-05-18/Mini-Call-Server
export PLAYSBC_VERSION=2.6.0

open -a Docker
until docker info >/dev/null 2>&1; do
  echo "Waiting for Docker Desktop..."
  sleep 5
done
  
```

Build Or Refresh Images

```

docker build -f docker/playsbc.Dockerfile \
  -t "playsbc:$PLAYSBC_VERSION" .

docker build -f docker/rtpengine.Dockerfile \
  -t "playsbc-rtpengine:$PLAYSBC_VERSION" .

docker image inspect "playsbc:$PLAYSBC_VERSION" >/dev/null
docker image inspect "playsbc-rtpengine:$PLAYSBC_VERSION" >/dev/null
  
```

Run Fast Application Regression

```

PYTHONPYCACHEPREFIX=/private/tmp/playsbc-pycache \
python3 tools/run_regression_suite.py \
  --skip-sipp-smoke \
  --all-b2bua-profiles \
  --timeout 420
  
```

Report: logs/reports/latest.html.

Run The Real-Address Docker Topology

```

PYTHONPYCACHEPREFIX=/private/tmp/playsbc-pycache \
python3 tools/run_real_topology.py --rebuild
  
```

This lane creates distinct core and peer Docker networks, a PlaySBC container, an RTPengine container, SIPp endpoints, RTP/RTCP helpers, and a combined evidence bundle.

Verify, Inspect, And Clean Up

```
docker compose -f docker-compose.topology.yml ps
docker compose -f docker-compose.topology.yml logs --tail=100 playsbc rtpengine

docker compose -f docker-compose.topology.yml down \
  --remove-orphans --volumes
```

To roll back, rebuild the previously accepted Git tag under a different immutable local tag and rerun the same profile. Do not retag an unverified image as the accepted release.

kind Canonical Regression Model

Canonical kind regression lane

One Docker-backed Kubernetes node, two active-active signalling/media replicas



Purpose And Prerequisites

The kind-playsbc context is the canonical 70-profile, active-active, HA-foundation regression lane. kind nodes are Docker containers, so Docker Desktop must remain running.

```
cd /Users/sudheerkumar/Documents/Codex/2026-05-18/Mini-Call-Server
export PLAYSBC_VERSION=2.6.0

open -a Docker
until docker info >/dev/null 2>&1; do sleep 5; done

kind get clusters
```

Create Or Recover The Cluster

Create the cluster only when kind get clusters does not list playsbc.

```
kind create cluster --name playsbc --wait 180s

kubectl config use-context kind-playsbc
kubectl create namespace playsbc \
  --dry-run=client -o yaml | kubectl apply -f -
kubectl config set-context --current --namespace=playsbc
kubectl get nodes
```

If the context exists but 127.0.0.1:<port> is refused, start Docker Desktop and the existing control-plane container first.

```
open -a Docker
until docker info >/dev/null 2>&1; do sleep 5; done
docker start playsbc-control-plane 2>/dev/null || true
kubectl --context kind-playsbc get nodes
```

Install Or Upgrade v2.6.0

Both repositories are mandatory. The active-active values create two StatefulSet replicas for PlaySBC and RTPengine plus Prometheus and Grafana.

```
helm upgrade --install playsbc \
  "https://github.com/sudheerkumarvatrapu/PlaySBC/releases/download/v${PLAYSBC_VERSION}/playsbc-${PLAYSBC_VERSION}.tgz" \
  --kube-context kind-playsbc \
  --namespace playsbc \
  --create-namespace \
  --atomic --wait --timeout 10m \
  -f configs/kubernetes/active-active-values.yaml \
  --set image.repository=ghcr.io/sudheerkumarvatrapu/playsbc \
  --set-string image.tag="${PLAYSBC_VERSION}" \
  --set image.pullPolicy=Always \
  --set rtpengine.enabled=true \
  --set rtpengine.image.repository=ghcr.io/sudheerkumarvatrapu/playsbc-rtpengine \
  --set-string rtpengine.image.tag="${PLAYSBC_VERSION}" \
  --set rtpengine.image.pullPolicy=Always \
  --set rtpengine.hostNetwork=false \
  --set playsbc.config.media_backend=rtpengine \
  --set-string playsbc.config.rtpengine_url=udp://playsbc-playsbc-rtpengine:2223 \
  --set observability.enabled=true \
  --set observability.prometheus.retention=31d \
  --set observability.prometheus.persistence.size=5Gi \
  --set observability.grafana.persistence.size=2Gi
```

Verify The Upgrade

```
helm --kube-context kind-playsbc -n playsbc status playsbc

kubectl --context kind-playsbc -n playsbc get pods -o wide
kubectl --context kind-playsbc -n playsbc get statefulset

kubectl --context kind-playsbc -n playsbc get statefulset \
  playsbc-playsbc playsbc-playsbc-rtpengine \
  -o custom-columns='NAME:.metadata.name,IMAGE:.spec.template.spec.containers[0].image,READY:.status.readyReplicas'

kubectl --context kind-playsbc -n playsbc exec \
  statefulset/playsbc-playsbc -- \
  python3 -c 'import mini_call_server as s; print(s.PLAYSBC_VERSION)'
```

Expected images are the two GHCR repositories with tag 2.6.0; the runtime version must print 2.6.0.

Run All 70 Profiles

```
PYTHONPYCACHEPREFIX=/private/tmp/playsbc-pycache \
python3 tools/run_k8s_regression_job.py \
  --all-profiles \
  --runner-image ghcr.io/sudheerkumarvatrapu/playsbc-k8s-regression:2.6.0 \
  --sipp-image ghcr.io/sudheerkumarvatrapu/playsbc-sipp:2.6.0 \
  --playsbc-image ghcr.io/sudheerkumarvatrapu/playsbc:2.6.0 \
  --rtpengine-image ghcr.io/sudheerkumarvatrapu/playsbc-rtpengine:2.6.0 \
  --set-playsbc-image \
  --set-rtpengine-image \
  --no-load-playsbc-image \
  --no-load-rtpengine-image \
  --no-load-sipp-image \
  --kind-cluster playsbc
```

The launcher prints Regression progress: X/70. The final report is under logs/k8s-job/<run-id>/k8s-reports/latest.html.

Run Selected Profiles

```
PYTHONPYCACHEPREFIX=/private/tmp/playsbc-pycache \
python3 tools/run_k8s_regression_job.py \
  --profile rfc5359-call-hold-resume \
  --profile rtpengine-media \
  --runner-image ghcr.io/sudheerkumarvatrapu/playsbc-k8s-regression:2.6.0 \
  --sipp-image ghcr.io/sudheerkumarvatrapu/playsbc-sipp:2.6.0 \
  --playsbc-image ghcr.io/sudheerkumarvatrapu/playsbc:2.6.0 \
  --rtpengine-image ghcr.io/sudheerkumarvatrapu/playsbc-rtpengine:2.6.0 \
  --set-playsbc-image --set-rtpengine-image \
  --no-load-playsbc-image --no-load-rtpengine-image --no-load-sipp-image \
  --kind-cluster playsbc
```

Roll Back And Clean Up

```
helm --kube-context kind-playsbc -n playsbc history playsbc
helm --kube-context kind-playsbc -n playsbc rollback playsbc <REVISION> \
  --wait --timeout 10m

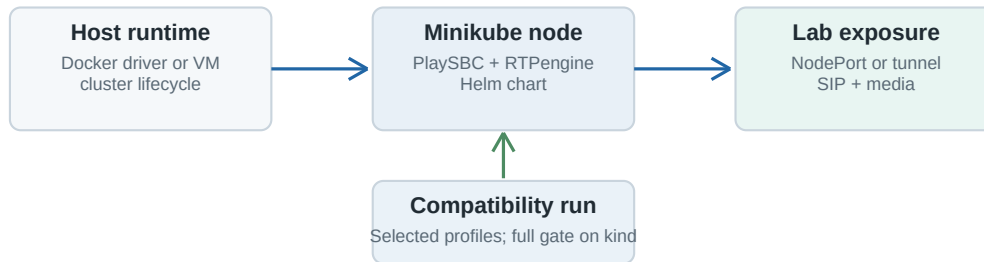
helm --kube-context kind-playsbc -n playsbc uninstall playsbc
kind delete cluster --name playsbc
```

If Helm is pending-upgrade after an interruption, roll back to the last deployed revision. If pods still reference the interrupted image, delete only those failed pods and let the rolled-back StatefulSet recreate them.

Minikube Compatibility Model

Minikube compatibility lane

Portable chart validation; kind remains the canonical full-regression environment



Purpose And Prerequisites

Minikube checks chart portability. The Docker driver requires Docker Desktop; VM drivers require their matching hypervisor. kind remains the release-gating local lane.

```

export PLAYSBC_VERSION=2.6.0
minikube start --driver=docker --profile playsbc

kubectl config use-context playsbc
kubectl create namespace playsbc \
  --dry-run=client -o yaml | kubectl apply -f -
kubectl config set-context --current --namespace=playsbc
  
```

Make Images Available

Use GHCR images with pull policy Always, or load locally built images into the profile. Do not combine a GHCR repository with pullPolicy: Never.

```

minikube -p playsbc image load "playsbc:$PLAYSBC_VERSION"
minikube -p playsbc image load "playsbc-rtpengine:$PLAYSBC_VERSION"
  
```

Install Or Upgrade

```

helm upgrade --install playsbc \
  "https://github.com/sudheerkumarvatrapu/PlaySBC/releases/download/v${PLAYSBC_VERSION}/playsbc-${PLAYSBC_VERSION}.tgz" \
  --kube-context playsbc \
  -n playsbc --create-namespace --atomic --wait --timeout 10m \
  -f configs/kubernetes/minikube-values.yaml \
  --set image.repository=ghcr.io/sudheerkumarvatrapu/playsbc \
  --set-string image.tag="$PLAYSBC_VERSION" \
  --set image.pullPolicy=Always \
  --set rtpengine.image.repository=ghcr.io/sudheerkumarvatrapu/playsbc-rtpengine \
  --set-string rtpengine.image.tag="$PLAYSBC_VERSION" \
  --set rtpengine.image.pullPolicy=Always
  
```

Verify, Test, Roll Back, And Clean Up

```
kubectl --context playsbc -n playsbc get pods,svc -o wide
helm --kube-context playsbc -n playsbc history playsbc

helm --kube-context playsbc -n playsbc rollback playsbc <REVISION> \
  --wait --timeout 10m

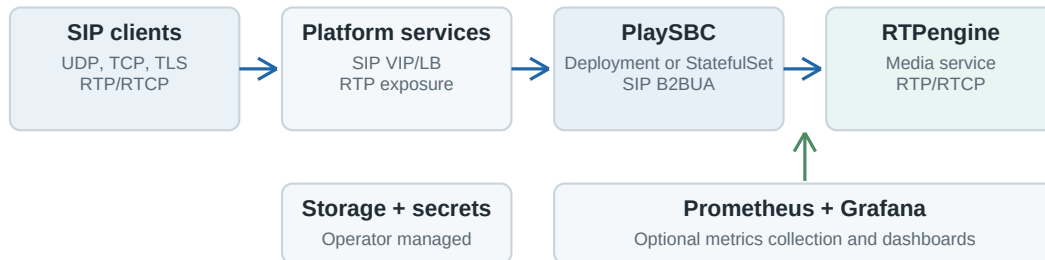
helm --kube-context playsbc -n playsbc uninstall playsbc
minikube delete --profile playsbc
```

Selected Kubernetes profiles may be run against this context after validating image availability and service reachability. Run the full release gate on kind.

Generic Helm Kubernetes Model

Generic Helm-managed Kubernetes

Operator supplies storage, exposure, identity, and network policy appropriate to the platform



Operator Responsibilities

The chart can render for another conformant Kubernetes cluster, but the operator owns the CNI, storage classes, LoadBalancer implementation, SIP/RTP firewall policy, certificates, identity, secrets, and capacity. An HTTP ingress controller does not replace SIP UDP/TCP/TLS or RTP/RTCP exposure.

Prepare And Render Before Applying

```

export PLAYSBC_VERSION=2.6.0
export KUBE_CONTEXT=<your-context>

kubectl --context "$KUBE_CONTEXT" get nodes
helm lint charts/playsbc

helm template playsbc \
  "https://github.com/sudheerkumarvatrapu/PlaySBC/releases/download/v${PLAYSBC_VERSION}/playsbc-${PLAYSBC_VERSION}.tgz" \
  --namespace playsbc \
  -f <platform-values.yaml> \
  --set image.repository=ghcr.io/sudheerkumarvatrapu/playsbc \
  --set-string image.tag="${PLAYSBC_VERSION}" \
  --set rtpengine.image.repository=ghcr.io/sudheerkumarvatrapu/playsbc-rtpengine \
  --set-string rtpengine.image.tag="${PLAYSBC_VERSION}" \
  >/tmp/playsbc-rendered.yaml
  
```

Inspect `/tmp/playsbc-rendered.yaml` for workload kind, replicas, images, Services, Secrets, storage, advertised IPs, and RTP range.

Install Or Upgrade

```

helm upgrade --install playsbc \
  "https://github.com/sudheerkumarvatrapu/PlaySBC/releases/download/v${PLAYSBC_VERSION}/playsbc-${PLAYSBC_VERSION}.tgz" \
  --kube-context "$KUBE_CONTEXT" \
  -n playsbc --create-namespace --atomic --wait --timeout 10m \
  -f <platform-values.yaml> \
  --set image.repository=ghcr.io/sudheerkumarvatrapu/playsbc \
  --set-string image.tag="${PLAYSBC_VERSION}" \
  --set rtpengine.image.repository=ghcr.io/sudheerkumarvatrapu/playsbc-rtpengine \
  --set-string rtpengine.image.tag="${PLAYSBC_VERSION}"
  
```

Verify, Test, Roll Back, And Clean Up


```
kubectl --context "$KUBE_CONTEXT" -n playsbc get \
  pods,svc,deploy,statefulset,pvc -o wide

helm --kube-context "$KUBE_CONTEXT" -n playsbc history playsbc
helm --kube-context "$KUBE_CONTEXT" -n playsbc rollback playsbc <REVISION> \
  --wait --timeout 10m

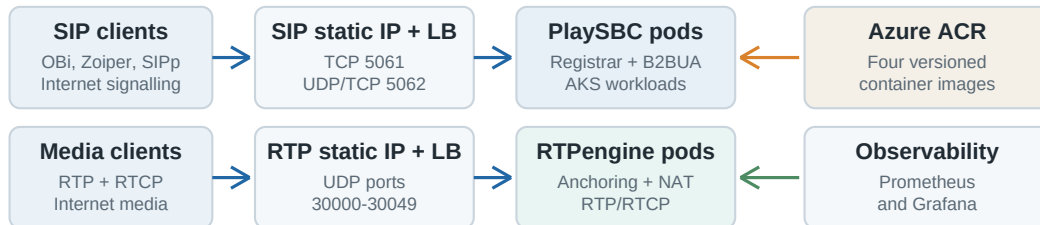
helm --kube-context "$KUBE_CONTEXT" -n playsbc uninstall playsbc
```

Run selected regression profiles first. Promote to a full suite only after platform-specific SIP and RTP networking is verified.

Azure AKS Administration and Deployment Model

Azure AKS public lab

Separate static SIP and RTP addresses; managed identity controls Network RG resources



AKS managed identity has Network Contributor on the network resource group

Purpose And Required Resources

The AKS lane validates Azure identity, ACR, static public IPs, Azure Load Balancers, public SIP/RTP reachability, cloud regression, and internet real-device calls.

- AKS resource group and cluster.
- Network resource group with separate static SIP and RTP public IP resources.
- Azure Container Registry containing all four v2.6.0 images.
- Network Contributor permission for the AKS identity on the network resource group.
- Firewall/source-range policy for SIP, health, and RTP ports.

Set Variables And Register Providers

```

export PLAYSBC_VERSION=2.6.0
export LOCATION=eastus
export AKS_RG=playsbc-aks-rg
export NETWORK_RG=playsbc-network-rg
export AKS_NAME=playsbc-aks
export ACR_NAME=playsbcacr$RANDOM
export SIP_PIP_NAME=playsbc-sip-pip
export RTP_PIP_NAME=playsbc-rtp-pip

for PROVIDER in Microsoft.ContainerService Microsoft.ContainerRegistry \
  Microsoft.Network Microsoft.Compute Microsoft.ManagedIdentity; do
  az provider register --namespace "$PROVIDER"
done

az group create -n "$AKS_RG" -l "$LOCATION"
az group create -n "$NETWORK_RG" -l "$LOCATION"
  
```

Create ACR And AKS

```
az acr create -g "$AKS_RG" -n "$ACR_NAME" --sku Basic

az aks create \
  -g "$AKS_RG" -n "$AKS_NAME" -l "$LOCATION" \
  --tier free --node-count 1 --node-vm-size Standard_D2as_v7 \
  --load-balancer-sku standard --attach-acr "$ACR_NAME" \
  --generate-ssh-keys

az aks get-credentials \
  -g "$AKS_RG" -n "$AKS_NAME" --overwrite-existing
kubectl get nodes
```

Create Static Public IPs And Grant Identity Access

```
az network public-ip create \
  -g "$NETWORK_RG" -n "$SIP_PIP_NAME" -l "$LOCATION" \
  --sku Standard --allocation-method Static

az network public-ip create \
  -g "$NETWORK_RG" -n "$RTP_PIP_NAME" -l "$LOCATION" \
  --sku Standard --allocation-method Static

AKS_OBJECT_ID=$(az aks show -g "$AKS_RG" -n "$AKS_NAME" \
  --query identity.principalId -o tsv)
NETWORK_RG_ID=$(az group show -n "$NETWORK_RG" --query id -o tsv)

az role assignment create \
  --assignee-object-id "$AKS_OBJECT_ID" \
  --assignee-principal-type ServicePrincipal \
  --role "Network Contributor" \
  --scope "$NETWORK_RG_ID"
```

This role assignment is persistent for the current managed identity and network resource group. Recreate it when either resource is recreated with a new identity or scope.

Import And Verify All Four Images

```
for IMAGE in playsbc playsbc-rtpengine playsbc-k8s-regression playsbc-sipp; do
  az acr import --name "$ACR_NAME" \
    --source "ghcr.io/sudheerkumarvatrapu/$IMAGE:$PLAYSBC_VERSION" \
    --image "$IMAGE:$PLAYSBC_VERSION" --force

  az acr repository show --name "$ACR_NAME" \
    --image "$IMAGE:$PLAYSBC_VERSION" -o none
done
```

Install Or Upgrade AKS

```
export ACR_LOGIN_SERVER=$(az acr show -g "$AKS_RG" -n "$ACR_NAME" \
  --query loginServer -o tsv)
export NODE_RG=$(az aks show -g "$AKS_RG" -n "$AKS_NAME" \
  --query nodeResourceGroup -o tsv)
export SIP_PUBLIC_IP=$(az network public-ip show -g "$NETWORK_RG" \
  -n "$SIP_PIP_NAME" --query ipAddress -o tsv)
export RTP_PUBLIC_IP=$(az network public-ip show -g "$NETWORK_RG" \
  -n "$RTP_PIP_NAME" --query ipAddress -o tsv)

helm upgrade --install playsbc \
  "https://github.com/sudheerkumarvatrapu/PlaySBC/releases/download/v${PLAYSBC_VERSION}/playsbc-${PLAYSBC_VERSION}.tgz" \
  -n playsbc --create-namespace --atomic --wait --timeout 15m \
  -f configs/kubernetes/aks-values.yaml \
  --set cloud.azure.nodeResourceGroup="$NODE_RG" \
  --set cloud.azure.sip.public.publicIPResourceGroup="$NETWORK_RG" \
  --set cloud.azure.sip.public.publicIPName="$SIP_PIP_NAME" \
  --set cloud.azure.media.public.enabled=true \
  --set cloud.azure.media.public.publicIPResourceGroup="$NETWORK_RG" \
  --set cloud.azure.media.public.publicIPName="$RTP_PIP_NAME" \
  --set image.repository="$ACR_LOGIN_SERVER/playsbc" \
  --set-string image.tag="$PLAYSBC_VERSION" \
  --set rtpengine.image.repository="$ACR_LOGIN_SERVER/playsbc-rtpengine" \
  --set-string rtpengine.image.tag="$PLAYSBC_VERSION" \
  --set-string rtpengine.advertisedIP="$RTP_PUBLIC_IP" \
  --set-string playsbc.config.sip_advertised_ip="$SIP_PUBLIC_IP" \
  --set-string playsbc.config.b2bua_advertised_ip="$SIP_PUBLIC_IP"
```

Verify Azure Services And Runtime

```
kubectl -n playsbc get pods -o wide
kubectl -n playsbc get svc \
  playsbc-playsbc-azure-sip-public \
  playsbc-playsbc-azure-rtp-public -o wide

kubectl -n playsbc describe svc \
  playsbc-playsbc-azure-sip-public | sed -n '/Events:$/, $p'

kubectl -n playsbc get deployment, statefulset \
  -o custom-columns='KIND:.kind, NAME:.metadata.name, IMAGES:.spec.template.spec.containers[*].image'
```

A pending external IP is not solved by restarting PlaySBC. Check Service events, static public-IP names, resource groups, and managed identity permission. Wait for both ingress addresses before registration, calls, or AKS regression.

Run AKS Regression: 12-Profile Readiness Suite

```
PYTHONPYCACHEPREFIX=/tmp/playsbc-pycache \
python3 tools/run_aks_regression_job.py \
  --aks-profiles \
  --runner-image "$ACR_LOGIN_SERVER/playsbc-k8s-regression:$PLAYSBC_VERSION" \
  --runner-image-pull-policy Always \
  --sipp-image "$ACR_LOGIN_SERVER/playsbc-sipp:$PLAYSBC_VERSION" \
  --sipp-image-pull-policy Always \
  --playsbc-image "$ACR_LOGIN_SERVER/playsbc:$PLAYSBC_VERSION" \
  --rtppengine-image "$ACR_LOGIN_SERVER/playsbc-rtppengine:$PLAYSBC_VERSION" \
  --set-playsbc-image --set-rtppengine-image \
  --aks-load-balancer-wait-timeout 1200 \
  --job-timeout 3600
```

Download `latest-aks-regression.tgz` immediately because Cloud Shell storage can be ephemeral.

Credential Recovery

Use the normal CLI first. If the installed CLI selects an unsupported API version, retrieve the credential payload with an explicitly supported API version.

```
az aks get-credentials \
  -g "$AKS_RG" -n "$AKS_NAME" --overwrite-existing
```

Fallback:

```
export SUB_ID=$(az account show --query id -o tsv)
mkdir -p ~/.kube

az rest --method post \
  --url "https://management.azure.com/subscriptions/$SUB_ID/resourceGroups/$AKS_RG/providers/Microsoft.ContainerService/managedClusters/$AKS_NAME/listClusterUsers" \
  --query 'kubeconfigs[0].value' -o tsv \
  | base64 -d > ~/.kube/config

chmod 600 ~/.kube/config
kubectl get pods -n playsbc
```

Roll Back And Clean Up

Resource-group deletion is asynchronous. A stale kube context may show cached objects briefly and later fail DNS/API access after the control plane is removed.

```
helm -n playsbc history playsbc
helm -n playsbc rollback playsbc <REVISION> --wait --timeout 15m

export SUB_ID=$(az account show --query id -o tsv)
for RG in "$AKS_RG" "$NETWORK_RG"; do
  if [ "$(az group exists --subscription "$SUB_ID" --name "$RG")" = true ]; then
    az group delete --subscription "$SUB_ID" \
      --name "$RG" --yes --no-wait
  fi
done
```

Local kind Real-Device Model

Local kind real-device lane

Dedicated cluster exposes the Mac LAN address; never reuse the regression cluster



Validated Baseline

Endpoint	User	Password	Transport
OBi1022	1001	secret-password	UDP
Zoiper	1002	secret-password	UDP

This lane uses a separate `playsbc-real-device` cluster. PlaySBC and RTPengine advertise the Mac LAN IPv4 and expose SIP plus RTP/RTCP one-to-one through kind port mappings.

Determine The LAN Address And Create The Cluster

```

export PLAYSBC_VERSION=2.6.0
export REAL_DEVICE_CLUSTER=playsbc-real-device
export REAL_DEVICE_CONTEXT=kind-playsbc-real-device
export LAN_IF=$(route -n get default | awk '/interface:/{print $2; exit}')
export LAN_IP=$(ipconfig getifaddr "$LAN_IF")
: "${LAN_IP:?Could not determine Mac LAN IPv4}"

kind create cluster \
  --name "$REAL_DEVICE_CLUSTER" \
  --config configs/kubernetes/kind-real-device-cluster.yaml \
  --wait 180s
  
```

Install Or Upgrade

```

helm upgrade --install playsbc \
  "https://github.com/sudheerkumarvatrapu/PlaySBC/releases/download/v${PLAYSBC_VERSION}/playsbc-${PLAYSBC_VERSION}.tgz" \
  --kube-context "$REAL_DEVICE_CONTEXT" \
  -n playsbc --create-namespace --atomic --wait --timeout 5m \
  -f configs/kubernetes/kind-real-device-values.yaml \
  --set-string localRealDevice.lanIPv4="$LAN_IP" \
  --set image.repository=ghcr.io/sudheerkumarvatrapu/playsbc \
  --set-string image.tag="$PLAYSBC_VERSION" \
  --set rtpengine.image.repository=ghcr.io/sudheerkumarvatrapu/playsbc-rtpengine \
  --set-string rtpengine.image.tag="$PLAYSBC_VERSION" \
  --set-string playsbc.config.sip_advertised_ip="$LAN_IP" \
  --set-string playsbc.config.b2bua_advertised_ip="$LAN_IP" \
  --set-string rtpengine.advertisedIP="$LAN_IP"
  
```

The TLS lane also requires Secret `playsbc-real-device-tls`. Validate UDP registration and calls before troubleshooting device certificate trust.

Verify And Configure Endpoints

```
kubectl --context "$REAL_DEVICE_CONTEXT" -n playsbc get pods -o wide
python3 tools/check_kind_real_device_lab.py \
  --context "$REAL_DEVICE_CONTEXT" \
  --cluster "$REAL_DEVICE_CLUSTER" \
  --lan-ip "$LAN_IP" \
  --expected-version "$PLAYSBC_VERSION"
```

- Proxy and registrar: \$LAN_IP.
- UDP/TCP SIP port: 5062.
- TLS SIP port: 5061 after certificate trust is configured.
- OBi1022: user 1001; Zoiper: user 1002.
- Outbound proxy: blank for this baseline.
- Media: RTP/AVP G.711 with RTPengine range 30000-30049/UDP.

Call And Capture Acceptance

- Both users register after the digest challenge.
- 1001 -> 1002 and 1002 -> 1001 ring, answer, and tear down cleanly.
- Both parties hear audio in both directions.
- RTPengine evidence reports both media directions and RTCP where endpoints generate it.

```
PYTHONPYCACHEPREFIX=/private/tmp/playsbc-pycache \
python3 tools/run_real_device_capture.py \
  --context "$REAL_DEVICE_CONTEXT" \
  --namespace playsbc \
  --duration 120 \
  --capture-image nicolaka/netshoot:latest
```

The output bundle contains one combined packet capture and synchronized workload logs. Interrupt once with **Ctrl+C**; the collector performs graceful finalization.

Monitor, Roll Back, And Clean Up

```
kubectl --context "$REAL_DEVICE_CONTEXT" -n playsbc logs -f \
  -l app.kubernetes.io/instance=playsbc \
  --all-containers=true --prefix --max-log-requests=10 --since=10m \
  | grep -aE 'SIP (INVITE|ACK|BYE|CANCEL)|SIP TX response|SIP response|SDP SUMMARY|RTPENGINE|RTP packet|RTCP|1001|1002'

helm --kube-context "$REAL_DEVICE_CONTEXT" -n playsbc history playsbc
helm --kube-context "$REAL_DEVICE_CONTEXT" -n playsbc rollback playsbc <REVISION> \
  --wait --timeout 5m

helm --kube-context "$REAL_DEVICE_CONTEXT" -n playsbc uninstall playsbc
kind delete cluster --name "$REAL_DEVICE_CLUSTER"
```

Helm Configuration and Lifecycle

Group	Administrative Responsibility
image / rtpengine.image	Repository, immutable tag, and pull policy.
playsbc.config	SIP ports, advertised addresses, RTP range, media backend, TLS, and policy.
rtpengine	Enablement, control URL, RTP range, host networking, and advertised media IP.
cloud.azure	Static public-IP names, network resource group, and LB exposure.
observability	Prometheus/Grafana enablement, persistence, and retention.
authSecret	Lab digest users; production secret management is not provided.

```
helm -n playsbc list
helm -n playsbc get values playsbc -a
helm -n playsbc history playsbc
helm -n playsbc rollback playsbc <REVISION> --wait --timeout 10m

kubectl -n playsbc rollout status deployment/playsbc-playsbc --timeout=240s
kubectl -n playsbc rollout status deployment/playsbc-playsbc-rtpengine --timeout=240s
```

Verify configured images and the runtime-reported PlaySBC version after every upgrade.

Operations and Observability

Image and Version Verification

```
kubectl -n playsbc get deployment,statefulset \
-l 'app.kubernetes.io/instance=playsbc,app.kubernetes.io/name in (playsbc,playsbc-rtpengine)' \
-o custom-columns='KIND:.kind,NAME:.metadata.name,IMAGES:.spec.template.spec.containers[*].image'

kubectl -n playsbc exec deployment/playsbc-playsbc -- \
python3 -c 'import mini_call_server as s; print(s.PLAYSBC_VERSION)'
```

Health and Metrics

```
kubectl -n playsbc exec deployment/playsbc-playsbc -- \
python3 -c "import urllib.request; print(urllib.request.urlopen('http://127.0.0.1:8080/readyz').read().decode())"

kubectl -n playsbc port-forward deployment/playsbc-playsbc-grafana 3000:3000
kubectl -n playsbc port-forward deployment/playsbc-playsbc-prometheus 9090:9090
```

Grafana visualizes Prometheus data. When a panel looks wrong, inspect the Prometheus query and raw PlaySBC metrics before changing dashboard colors or thresholds.

Regression and Evidence Administration

The canonical local Kubernetes suite contains 70 profiles. The launcher prints live progress and inhibits macOS sleep for the lifetime of a non-dry-run Job.

```
PYTHONPYCACHEPREFIX=/private/tmp/playsbc-pycache \  
python3 tools/run_k8s_regression_job.py \  
  --all-profiles \  
  --runner-image ghcr.io/sudheerkumarvatrapu/playsbc-k8s-regression:2.6.0 \  
  --sipp-image ghcr.io/sudheerkumarvatrapu/playsbc-sipp:2.6.0 \  
  --playsbc-image ghcr.io/sudheerkumarvatrapu/playsbc:2.6.0 \  
  --rtengine-image ghcr.io/sudheerkumarvatrapu/playsbc-rtengine:2.6.0 \  
  --set-playsbc-image --set-rtengine-image \  
  --no-load-playsbc-image --no-load-rtengine-image --no-load-sipp-image \  
  --kind-cluster playsbc
```

Evidence Contract

Artifact	Purpose
latest.html	Profile verdicts and links to retained evidence.
sipmsg.log	Combined readable SIP messages.
capture.pcap	Combined non-load signalling/media/network packet evidence.
pcap-legs.json	Expected roles, packet counts, SIP events, and B2BUA INVITE legs.
log.sip / log.media	Protocol and media decisions.
archive	Portable evidence bundle for review outside the cluster.

Open Clickable Evidence

Do not open `latest.html` with `file://` in a restricted browser. Start the localhost-only evidence viewer from the repository and keep that terminal running while reviewing the report.

```
python3 tools/serve_regression_report.py \  
  /absolute/path/to/k8s-regression-<run-id>/k8s-reports/latest.html
```

The printed `http://127.0.0.1:8765/...` report renders text evidence safely in the browser. PCAP links show the retained file checksum and exact Wireshark command.

Security, HA, and Recovery

Security Baseline

- Use TLS secrets and trusted certificates for TLS labs.
- Keep credentials outside source-controlled values in any non-lab environment.
- Restrict public SIP, health, and RTP exposure with firewall and network policy controls.
- Use immutable image tags and verify digests before promotion.
- Review logs, transcripts, and media before sharing evidence.

HA Baseline

The public chart and suite include active-active, node-drain, shared-state, and failure-injection foundations. They are lab evidence, not a claim of lossless carrier-grade failover or RTPengine media-session migration.

Recovery Sequence

- Confirm kube context, node readiness, and current Helm revision.
- Inspect pod state and events before restarting workloads.
- Verify images, ConfigMap values, advertised addresses, and RTP range.
- Roll back Helm when a known-good revision is safer than an in-place repair.
- Collect logs and evidence before deleting a failed pod when possible.

Troubleshooting Guide

Symptom	Likely Cause	Action
127.0.0.1 refused	Docker/kind API stopped	Start Docker and control-plane; verify context.
InvalidImageName	Empty ACR/version variable	Export variables and verify the tag before Helm.
ImagePullBackOff	Missing tag or ACR auth	Check ACR tag and AKS pull role.
Azure IP pending	Static IP or identity permission	Describe Service; verify IP names and Network Contributor.
REGISTER absent	Wrong IP/port/transport	Test OPTIONS; inspect LB and endpoint profile.
One-way/no audio	Wrong RTP IP/range or NAT	Compare SDP, RTP LB, RTPengine interface, and PCAP.
Long-run time gap	Host sleep or shell disconnect	Use caffeinate-enabled launcher and preserve remote Job.
PCAP lacks a leg	Missing/empty role capture	Inspect pcap-legs.json; v2.6.0 fails the evidence gate.

Capacity and Product Boundaries

What v2.6.0 Can Claim

- Repeatable lab registration, signalling, media, AI-foundation, HA-foundation, observability, and evidence scenarios.
- Validated OBi1022/Zoiper calls in documented AKS and local-LAN topologies.
- A structured path for measuring behavior through Docker, Kubernetes, AKS, and real devices.

What It Does Not Claim

- Production certification for 300,000 registered devices or 2,500 concurrent calls.
- Lossless mid-call migration for PlaySBC or RTPengine failures.
- Complete RFC 3261, carrier interconnect, STIR/SHAKEN, lawful intercept, emergency calling, or regulatory certification.
- Production distributed state, multi-zone recovery, or commercial support SLA.

Capacity statements must come from repeatable tests that publish topology, image digests, workload, duration, CPU, memory, packet rate, CPS, registrations per second, media quality, and failure behavior.

Command Reference

Inspect Workloads

```
kubectl -n playsbc get pods,svc,deploy,statefulset -o wide
```

Restart Observability

```
kubectl -n playsbc rollout restart deployment/playsbc-playsbc-grafana
kubectl -n playsbc rollout restart deployment/playsbc-playsbc-prometheus
kubectl -n playsbc rollout status deployment/playsbc-playsbc-grafana --timeout=180s
kubectl -n playsbc rollout status deployment/playsbc-playsbc-prometheus --timeout=180s
```

Monitor Calls and Media

```
kubectl -n playsbc logs -f \
  -l app.kubernetes.io/instance=playsbc \
  --all-containers=true --prefix --max-log-requests=10 --since=10m \
  | grep -aE 'SIP (INVITE|ACK|BYE|CANCEL)|SIP TX response|SIP response|SDP SUMMARY|RTPENGINE|RTP packet|RTCP|1001|1002'
```

Release Artifact Verification

```
shasum -a 256 -c release/helm/playsbc-2.6.0.tgz.sha256
helm lint charts/playsbc
helm template playsbc charts/playsbc \
  -f configs/kubernetes/active-active-values.yaml >/tmp/playsbc.yaml
```

Canonical References

- docs/PRODUCT_GUIDE.md - editable source for this administration guide.
- docs/KUBERNETES_LOCAL.md - local topology and network model.
- docs/RTPENGINE_LOCAL.md - RTPengine design and focused media behavior.
- docs/OBSERVABILITY.md - Grafana and Prometheus operations.
- docs/AI_VOICE_GATEWAY.md - public AI lab baseline.
- release/RELEASE_NOTES_2.6.0.md - immutable release record.